Appendix 1

(c)

```
import numpy as np

def rodrigues_R(s, phi):
    """Rotation matrix for angle phi about axis vector s (3,)."""
    s = np.asarray(s, dtype=float)
    s = s / np.linalg.norm(s)  # normalize axis
    sx, sy, sz = s
    K = np.array([[0,   -sz,  sy],
                  [sz,   0,  -sx],
                  [-sy, sx,   0]], dtype=float)
    I = np.eye(3)
    return I + np.sin(phi) * K + (1 - np.cos(phi)) * (K @ K)

# example: axis, angle
s = np.array([1.0, 2.0, 3.0])
phi = 0.7

R = rodrigues_R(s, phi)
print("R=\n", R)

# eigenvalues / eigenvectors
w, V = np.linalg.eig(R)
print("\neigenvalues:\n", w)

# axis direction should be eigenvector with eigenvalue about 1
u = s / np.linalg.norm(s)
axis_err = np.linalg.norm(R @ u - u)
print("\n||Ru - u|| =", axis_err)
```

```python
# trace identity check
lhs = np.cos(phi)
rhs = 0.5 * (np.trace(R) - 1)
print("\ncos(phi) =", lhs)
print("0.5*(trace(R)-1) =", rhs)
print("abs diff =", abs(lhs - rhs))

# show points before/after
points = np.array([[1,0,0],
                   [0,1,0],
                   [0,0,1],
                   [1,1,1]], dtype=float)

rot = (R @ points.T).T
print("\npoints: x -> Rx")
for x, y in zip(points, rot):
    print(x, "->", y)
```

```
R=
 [[ 0.78163917 -0.48292928  0.3947398 ]
 [ 0.55011723  0.83203013 -0.0713925 ]
 [-0.29395788  0.27295634  0.91601507]]

eigenvalues:
 [0.76484219+0.64421769j 0.76484219-0.64421769j 1.        +0.j        ]

||Ru - u|| = 1.5700924586837752e-16

cos(phi) = 0.7648421872844885
0.5*(trace(R)-1) = 0.7648421872844884
abs diff = 1.1102230246251565e-16

points: x -> Rx
[1. 0. 0.] -> [ 0.78163917  0.55011723 -0.29395788]
[0. 1. 0.] -> [-0.48292928  0.83203013  0.27295634]
[0. 0. 1.] -> [ 0.3947398  -0.0713925   0.91601507]
[1. 1. 1.] -> [0.69344969 1.31075487 0.89501353]
```

(d)

```
import numpy as np

def axis_angle_from_R(R, eps=1e-12):
    """
    Given a 3x3 rotation matrix R (orthogonal, det ~ 1),
    return (s, phi) where s is the unit axis and phi is the rotation angle in [0, pi].

    Uses:
      cos(phi) = (trace(R) - 1)/2
      R - R^T = 2 sin(phi) * hat(s)
    Handles near-0 and near-pi angles with simple stable fallbacks.
    """
    R = np.asarray(R, dtype=float)

    # angle from trace
    c = (np.trace(R) - 1.0) / 2.0
    c = np.clip(c, -1.0, 1.0)
    phi = np.arccos(c)

    # near 0: axis is not unique; return a default axis
    if phi < 1e-8:
        return np.array([1.0, 0.0, 0.0]), 0.0

    # general case: use skew part to get axis
    if abs(np.sin(phi)) > 1e-8:
        A = (R - R.T) / (2.0 * np.sin(phi))  # should equal hat(s)
        s = np.array([A[2, 1], A[0, 2], A[1, 0]])
        s = s / np.linalg.norm(s)
        return s, phi

    # near pi: sin(phi) ~ 0, use diagonal-based recovery
    # For phi ~ pi, R   2 s s^T - I   =>   (R + I)/2   s s^T
    M = (R + np.eye(3)) / 2.0
    s = np.array([np.sqrt(max(M[0, 0], 0.0)),
                  np.sqrt(max(M[1, 1], 0.0)),
                  np.sqrt(max(M[2, 2], 0.0))])

    # choose signs using off-diagonals (if possible)
    if M[0, 1] < 0: s[1] = -s[1]
```

```
    if M[0, 2] < 0: s[2] = -s[2]
    # normalize (in case of small numerical drift)
    if np.linalg.norm(s) < eps:
        # fallback if degenerate
        s = np.array([1.0, 0.0, 0.0])
    else:
        s = s / np.linalg.norm(s)
    return s, phi


# quick test using your previous R (optional)
if __name__ == "__main__":
    R = np.array([[ 0.78163917, -0.48292928,  0.3947398 ],
                  [ 0.55011723,  0.83203013, -0.0713925 ],
                  [-0.29395788,  0.27295634,  0.91601507]])
    s, phi = axis_angle_from_R(R)
    print("axis s =", s)
    print("phi =", phi)
    print("cos(phi) from trace =", 0.5*(np.trace(R)-1))
    print("cos(phi) =", np.cos(phi))
```

```
axis s = [0.26726124 0.53452249 0.80178372]
phi = 0.7000000035461436
cos(phi) from trace = 0.764842185
cos(phi) = 0.764842185
```